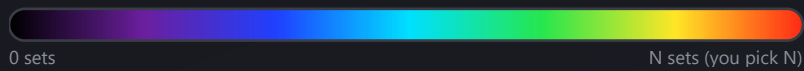


hotMuscles

A phone app for logging gym workouts. It shows how many sets each muscle got in the last 7 days as a colour heatmap on a body.



884

exercises

30

muscle regions

5

screens

4

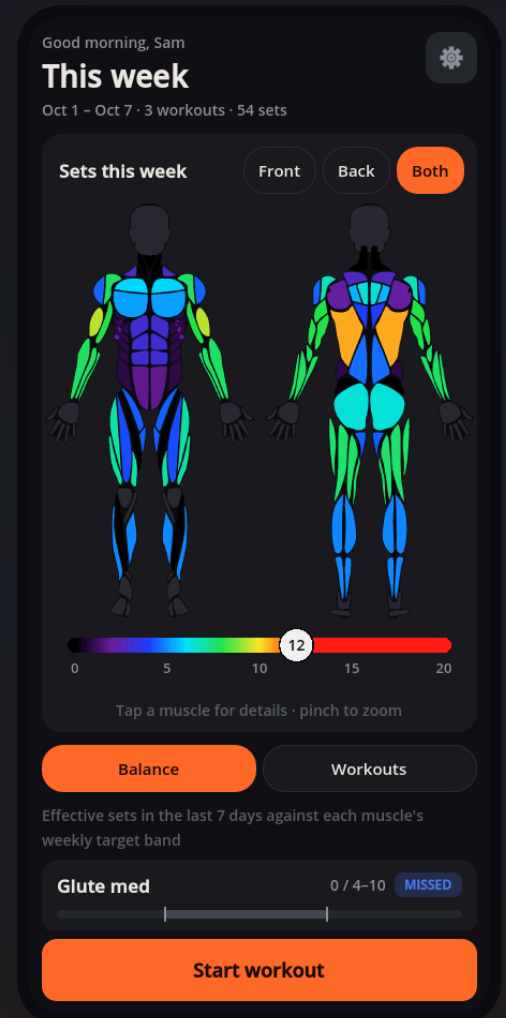
bodies × views

4 · 3

themes · gradients

8.1k

lines of app code



The home screen: this week's heat on front + back

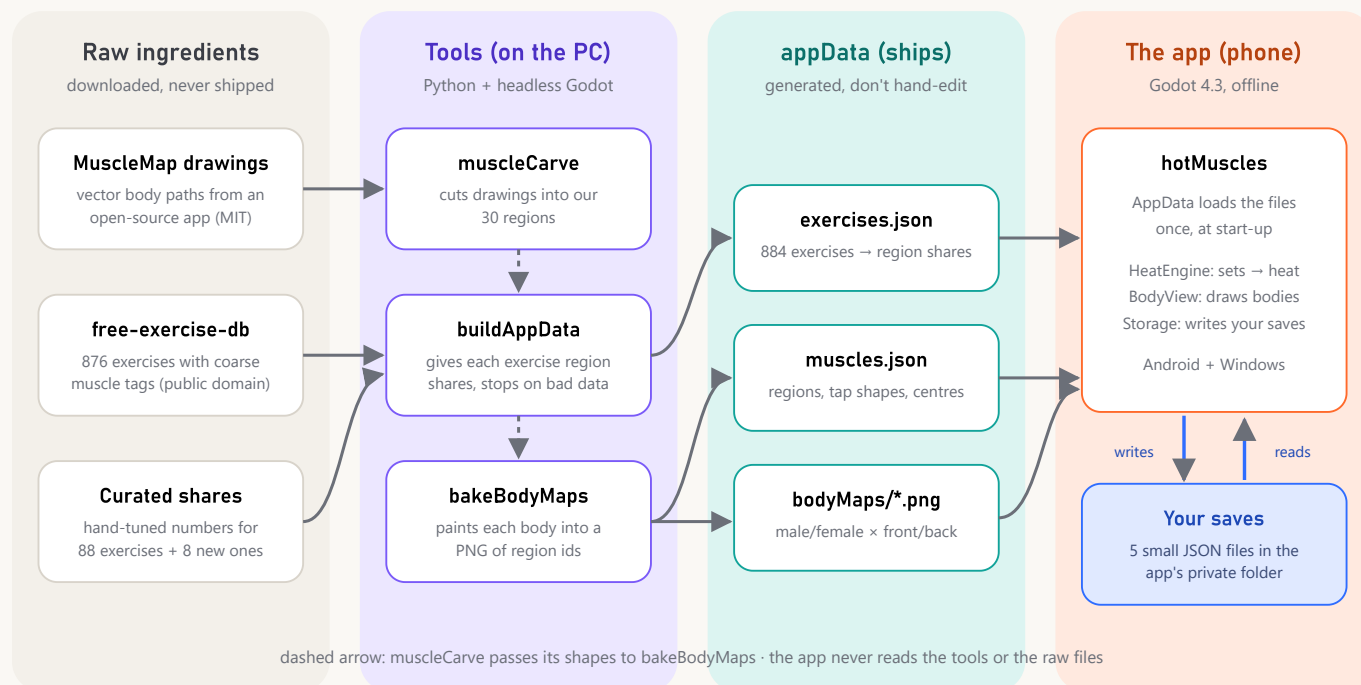
What's inside

1 Data flow · 2 App structure · 3 Screens and navigation · 4 Saved data · 5 Heat calculation · 6 Body rendering · 7 Behind the scenes

Built with Godot 4.3 · one codebase → Android apk + Windows exe · fully offline

Data flow: from source files to the app on your phone

Data moves left to right. Body drawings and an exercise list go in on the left. Python tools on the PC turn them into data files, which are packed into the app and read when it starts. The only thing the app writes is your own workout log, saved on the phone.



tools run on the PC only

Re-run them when a body drawing or an exercise share changes. If a share is out of range, an id is unknown, or a muscle can't be reached by any exercise, they stop and write nothing, so bad data can't reach the app.

appData generated, ships

Finished data files. The slow work is done once on the PC, so the phone only loads files and looks values up, and the app starts fast. The body PNGs are packed in exactly as made, never re-compressed.

saves written by the app

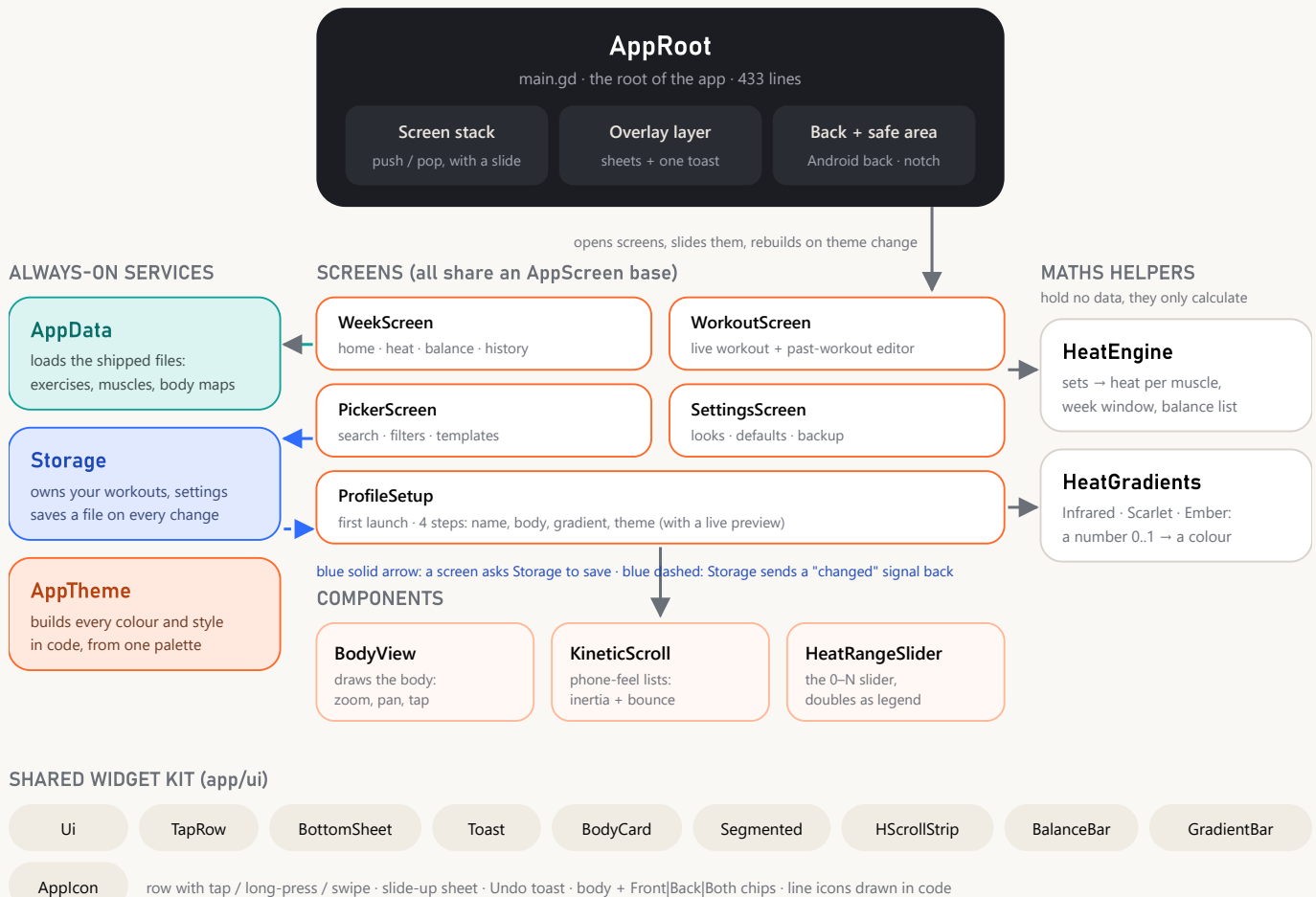
The only files the app writes. They are small, carry a version number, and survive a crash mid-save. One tap in Settings exports all of them as a backup.

Project folders and what they contain

FOLDER	SHIPS?	WHAT'S IN IT
app/	in the app	the app itself: <code>core/</code> data services + heat maths, <code>screens/</code> the five screens, <code>components/</code> the body view, scroller and slider, <code>ui/</code> small shared widgets, <code>main.gd</code> the root
appData/	in the app	the finished data files: <code>exercises.json</code> , <code>muscles.json</code> , <code>bodyMaps/</code> (the four body PNGs)
tools/	PC only	muscleCarve (cuts the regions), the appData builder and body-map baker, build scripts, the icon maker
tests/	PC only	the automatic checks and screenshot drivers
data/ · models/	PC only	the raw input files: carved shapes, MuscleMap source, upstream notes
docs/ · icons/	PC only	the app spec and this guide, launcher icon sources

App structure: main parts and how they connect

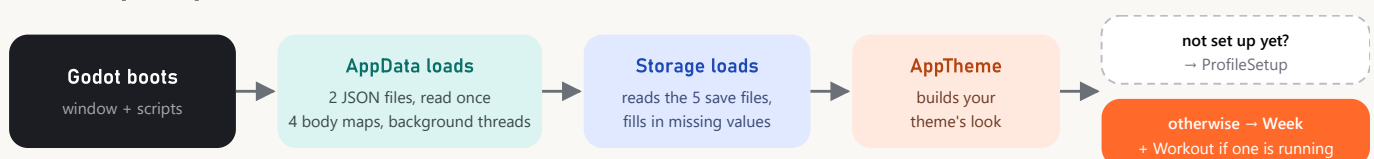
AppRoot (`main.gd`) is the top node of the app. It keeps the stack of open screens and the layer for sheets and toasts. Three services stay loaded the whole time: AppData holds the shipped files, Storage holds your saves, AppTheme holds the colours. Two helpers, HeatEngine and HeatGradients, only calculate. Screens are built from shared widgets in `ui/`.



Update loop: a screen asks Storage to change something. Storage writes the file and sends `changed("workouts")` (a Godot signal: a broadcast message). Every screen listening for it refreshes itself. Screens never call each other directly.

Theme change: AppTheme rebuilds every colour and style from one palette (page, surface, text, accent, cold, hot, ghost...). Then AppRoot tells every open screen to rebuild itself with the new colours.

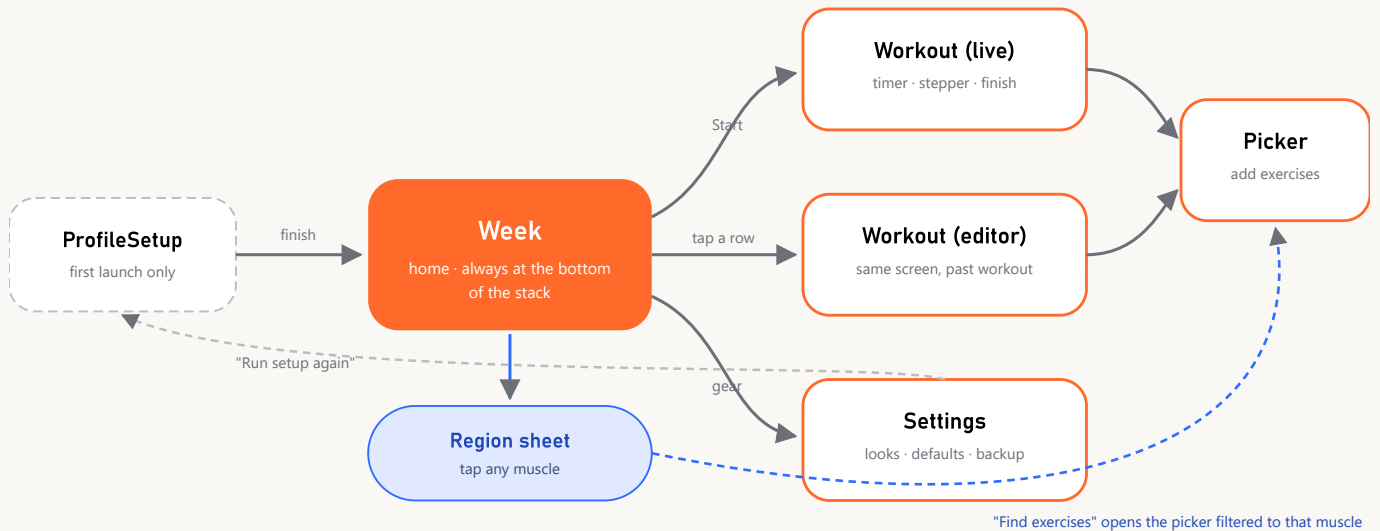
Start-up sequence



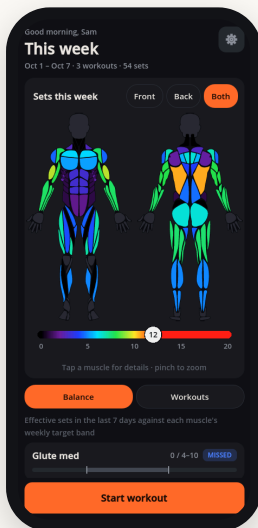
Limit: under 500 ms from the main scene to the first drawn frame. The build check measures it (more on page 8).

Screens and navigation between them

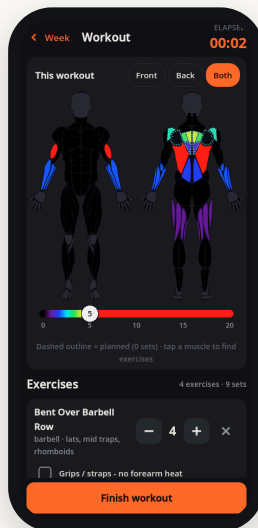
AppRoot keeps the open screens in a stack. Opening a screen slides it in on top, and Back slides the top one away. Week is always at the bottom. Bottom sheets (like the region sheet) are not screens: they slide up over the current screen from a separate overlay layer.



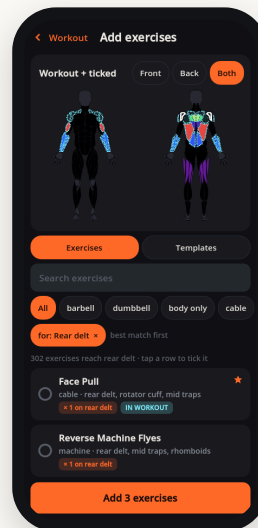
"Find exercises" opens the picker filtered to that muscle



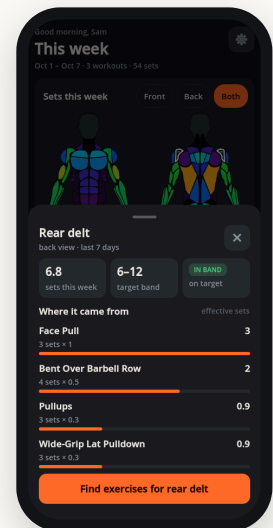
Week



Workout



Picker, filtered to rear delt



Region sheet

Logging one workout: what you tap and what gets saved

1 · Start workout

On Week. Storage creates the workout with a start time, and the Workout screen slides in.

2 · Add exercises

The picker opens. Ticked exercises show on the body as dashed outlines. Nothing is saved yet.

3 · Add

Storage saves all ticked exercises in one write. Each appears as a row with 0 sets (dashed).

4 · - / + sets

Storage saves on every tap, and the muscles light up with this workout's heat.

5 · Finish

Storage drops the 0-set rows and stamps the end time. You can save it as a template.

6 · Back on Week

The week's heat updates on the body and a toast confirms.

Back button: what it does, in order

1 A sheet is open: close it. 2 Ask the screen first: ProfileSetup steps back one step, the editor asks about unsaved changes. 3 Otherwise slide the top screen away. 4 Nothing left: leave the app (Escape on PC never quits).

Resume and edit: if you close the app mid-workout, AppRoot opens Week and then the running workout on top. Editing a past workout changes a copy until you press Save changes.

Saved data: the five save files and their fields

Each kind of data is saved in its own JSON file in the app's private folder. Storage reads all five files when the app starts and rewrites a file whenever its data changes. Every file is wrapped as `{"version": 1, "data": ...}`, so a future layout change can upgrade old files.

profile	profile.json	settings	settings.json	exercisePrefs	exercisePrefs.json
who you are and how the app looks:		display and default choices:		your choices, one entry per exercise:	
name	text	hideUntouched	yes / no	favourite	star in the picker
body	male female	newExerciseSets	0 or 1 (default 1)	hidden	hide from lists
gradient	infrared scarlet ember	defaultView	front back both	grips	remembered checkbox
theme	ember ocean forest light	rangeWeek	N for the week · 12	lastUsed	time, for sorting
setupDone	yes / no	rangeWorkout	N for a workout · 5		

workouts.json: workouts contain entries

A workout is a list of entries, and an entry is one exercise with its set count. `submitted` holds finished workouts, `current` holds the one in progress. `templates.json` reuses the same entry shape, which is why a template drops straight into a new workout.

workouts.json { submitted: [every finished workout], current: the running one, or null }

workout id · startedAt · endedAt (0 = still running)

entry exerciseld "Face_Pull" sets 3 grips false
sets 0 = planned, shown dashed, dropped on finish

entry exerciseld "Bent_Over_Barbell_Row" sets 4 grips true
grips ticked = straps used, so no forearm heat from this one

... more entries

templates.json
a list of { id, name, entries }
same entry shape, so a template drops straight into a workout

backup file
all five sections in one JSON, plus the app name, backupVersion, exportedAt
→ saved as a file and copied to the clipboard

Which screen writes which file

FILE	WRITTEN BY
<code>profile.json</code>	ProfileSetup (first launch) and Settings (name, body, gradient, theme)
<code>settings.json</code>	Settings (display and default choices) and the 0–N slider on the body card (rangeWeek, rangeWorkout)
<code>exercisePrefs.json</code>	Picker (favourite, hide) and Workout (the remembered grips tick)
<code>workouts.json</code>	Week (start a workout, delete one with Undo) and Workout (add exercises, set counts, finish, discard, edit a past workout)
<code>templates.json</code>	Week and Workout (save as template) and Picker (rename, delete)

Crash-safe writing: Storage writes `workouts.json.tmp` first and swaps it in only once it is complete, so a power cut mid-save leaves the old file whole. A file that can't be read is renamed `.bad` and only that section starts fresh.

Backup import: the app checks the file has every section with the right types, shows how many workouts and templates it holds, and asks you to confirm. It then saves a backup of your current data and replaces everything. It is all or nothing.

Heat calculation: from exercises to muscle colours

Each exercise in `exercises.json` lists the muscle regions it works and how much of a set counts for each (its **shares**). HeatEngine multiplies your sets by the shares and adds them up over the last 7 days. The total for each region is looked up on a colour gradient.

exercise record

exercises.json

id · name

"Ab_Roller" · Ab Roller

equipment · category

12 kinds · 7 kinds

forearmKind

none | grip | direct

curated

yes = hand-tuned shares

targets: one share per region (1.0 = a full set)

upperAbs

1.0

lowerAbs

0.8

serratus

0.4

lats

0.3

obliques

0.3

Where the shares come from

Automatic (788 exercises): the source database tags coarse muscle groups. `buildAppData.py` counts primary groups as 1.0 and secondary as 0.5, then splits each group over our finer regions (chest → lowerChest 0.7, upperChest 0.3). A region is capped at 1.0.

Curated (96): shares typed in by hand. **Stretching and cardio (137):** no heat.

Grips: for 106 exercises the forearm share is skipped if you tick "grips" (straps). 28 more train the forearms directly and always count.

$$\text{heat}[\text{muscle}] = \sum \text{sets} \times \text{share}$$

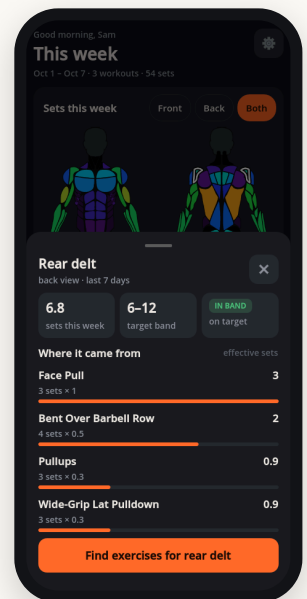
HeatEngine sums this over every entry of every workout finished in the last 7 × 24 hours

Worked example: rear delt (same numbers as the screenshot →)

Face Pull	3 sets × 1.0		3.0
Bent Over Barbell Row	4 sets × 0.5		2.0
Pullups	3 sets × 0.3		0.9
Wide-Grip Lat Pulldown	3 sets × 0.3		0.9
rear delt this week			6.8

band 6–12 → status "ok"

The face pull works the rear delt fully (1.0). The row and the lat exercises only assist it (0.5 and 0.3). Tapping a muscle opens the region sheet, which lists these same rows.



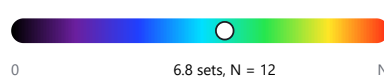
Region sheet for the rear delt

Last 7 days, no fading



The window is the 7 days before now, not the calendar week. A workout counts fully until it is 7 days old, then drops out.

Heat number to colour



HeatGradients turns $t = \text{sets} \div N$ (kept between 0 and 1) into a colour. The 0–N slider sets N; anything above N shows the top colour.

Status vs target band

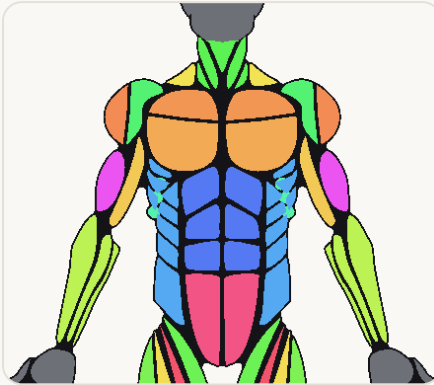
missed under ok over

Each region has a weekly band in sets: 6–12 for most, 10–20 for lats, glutes and hamstrings, 3–8 for small helpers. Under 0.5 sets is "missed". The balance list on Week puts the furthest from the band first.

30 regions in 7 groups: Back 6 · Shoulders 4 · Chest 3 · Arms 3 · Core 4 · Legs 9 · Neck 1. In the picker, tapping a muscle ranks exercises by $\text{share} + 0.5 \times \text{focus}$ (the part of the exercise that goes to that muscle). Favourites and exercises you have used before rank a little higher.

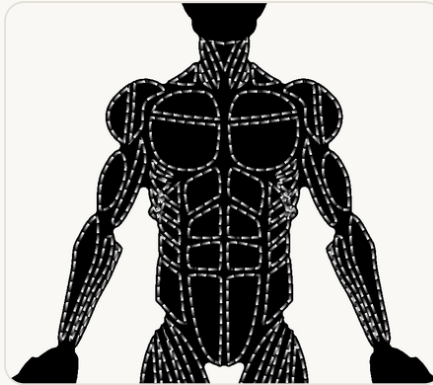
Body rendering: region map texture, palette and shader

Each body is drawn as one textured rectangle. The texture is not a picture. It is a region map: each colour channel stores a different number per pixel (like an ID pass in compositing). The shader `bodyMap.gdshader` reads those numbers and colours the body on every frame.



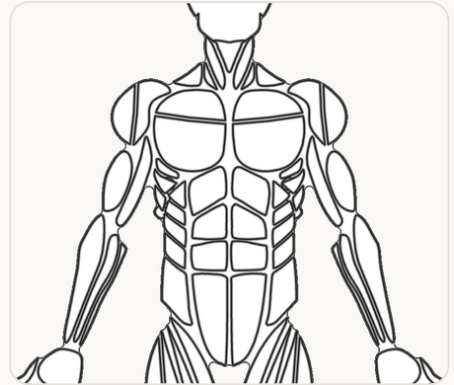
R · which region

the id of the muscle at this pixel: 0 outside the body, 1 seam between regions, 2 head, hands and feet, 3 and up one of the 30 regions. Shown here with a random colour per id.



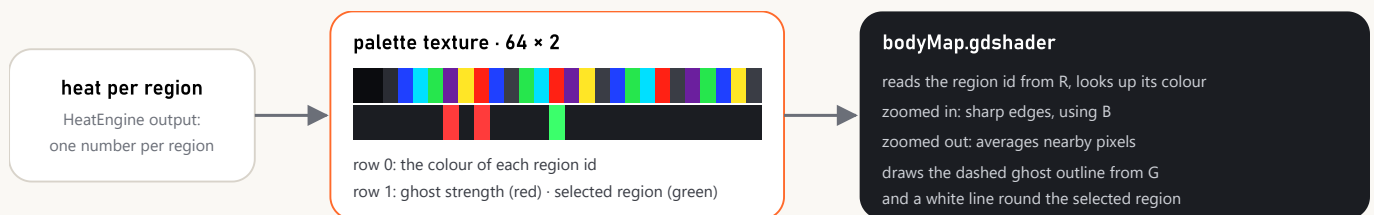
G · position along the outline

A number that runs along each muscle's edge. The shader uses it to draw the dashed "ghost" outline that marks exercises you planned but have not done yet.



B · distance to the edge

How far the pixel is from the nearest edge. The shader uses it to keep muscle edges sharp at any zoom level.



Colour update when the heat changes

BodyView works out the 30 new colours, writes them into the palette texture, and fades from the old colours to the new ones. Only the small palette changes. The big body texture is never touched, so an update is cheap.

Tap detection

A tap does not read the texture. BodyView tests it against the tap shapes stored in `muscles.json` (up to 32 corners each): box check, then point-in-polygon, top piece wins. A tap on a seam samples points around it. Pinch or wheel zooms 1-6x.

From a logged workout to colours on the Week screen

1 · Save

Storage writes `workouts.json` and sends the `changed("workouts")` signal.

2 · Count

WeekScreen hears it and asks HeatEngine for the week's heat: one number per region.

3 · Colour

BodyView turns each number into a colour with HeatGradients and writes it into the palette texture.

4 · Draw

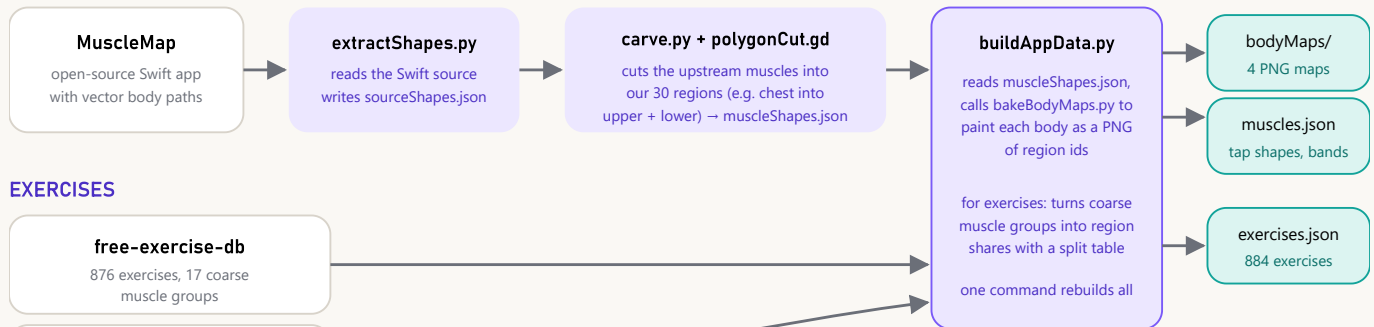
`bodyMap.gdshader` looks up every pixel's region id in the palette, so the GPU recolours the body.

Why it works this way: an earlier version drew tens of thousands of outline points per body on every frame. Now the GPU gets one rectangle and two small textures, and the phone builds no geometry at all. If a body map file is missing, the app shows an error instead of quietly drawing something else.

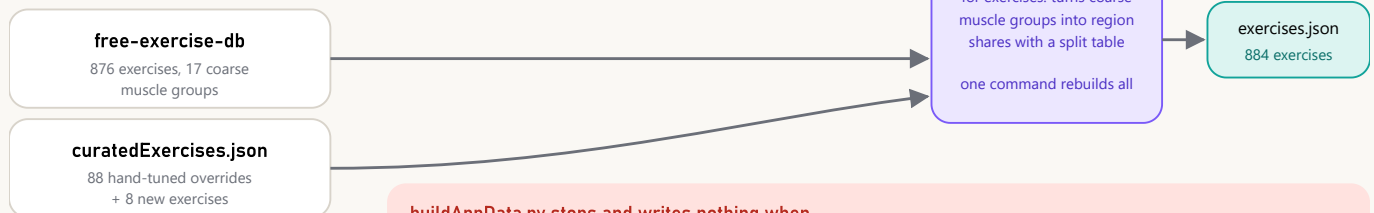
Behind the scenes: building the data and keeping it fast

Two jobs happen out of sight. Scripts on the PC build the data files once, so the phone has almost nothing to compute at start-up. And every build is checked against two time limits, so the app stays quick to open and smooth to use.

BODY SHAPES



EXERCISES



buildAppData.py stops and writes nothing when...

an exercise or region id is unknown · a share is outside 0–1 · any muscle can't be reached by some exercise

< 500 ms

from the main scene to the first drawn frame (first launch on a fresh PC: under 2 s, while the graphics driver prepares shaders once)

< 40 ms

for the slowest single frame during any everyday step (tap, scroll, open a screen). The aim is 33 ms (30 fps); v0.3 measured 27–32 ms at worst

How the app meets those limits

- ✓ **Long lists fill in over several frames.** Opening the picker builds the first 6 rows, then 3 more each frame, up to 40. Building all 40 at once took about 90 ms, a visible stutter. The rows off screen finish while the screen is still sliding in. The balance list on Week does the same: 4 rows, then 3 per frame.
- ✓ **Update rows, don't rebuild them.** Existing rows are changed or moved in place instead of being deleted and made again.
- ✓ **One write for many values.** Each file write costs about 4 ms, so "Add 3 exercises" and finishing setup each save everything in a single write.
- ✓ **Body maps load in the background.** AppData decodes the four PNGs on worker threads at launch, so the main thread stays free to draw the first frame.
- ✓ **Idle bodies sleep.** BodyView stops updating every frame when nothing on the body is animating.

Automatic checks cover the heat maths, saving and backups, each screen's flows with real numbers, a synthetic finger (tap, drag, swipe, sheets, back), list scrolling, start-up time, and the worst frame of every everyday step. The build check also opens each finished exe three times on a throwaway save folder.

Rebuild and ship rules

```
python tools/appData/buildAppData.py
```

Rebuilds every file in appData/. The folders data/, models/, tools/, tests/, docs/ and icons/ never go into a build, and the build check refuses a build that contains any of them.